Positive Square Energy of Every Hexacyclic Graph

AI-generated manuscript; no human authorship is claimed

12 August 2026

Abstract

Let G be a finite simple connected graph with n vertices and $n+5$ edges. We prove that the sum $s^+(G)$ of the squares of its positive adjacency eigenvalues is at least n. Cyclomatic-rank additivity splits the proof into graphs with at least two positive-rank cyclic blocks and graphs with exactly one. The first branch is covered by an owner-exact multiblock ledger. In the second branch, suppression gives one of 1198 rank-six multigraph kernels, with order distribution $1, 4, 26, 84, 216, 314, 325, 162, 66$ on two through ten branch vertices. Exact finite owners cover the complete kernel partition; exact path elimination promotes their physical parity frontiers to arbitrary simple subdivisions, and one-vertex additivity includes arbitrary rooted trees. A fail-closed all-connected master authenticates both branches, checks that their scopes are disjoint and exhaustive, and preserves the nonstrict conclusion. The computational layer uses committed exact-execution receipts; their byte authentication is distinguished explicitly from a fresh independent replay.

1 Statement and scope

All graphs in the spectral statements are finite, simple, and undirected. For the adjacency eigenvalues $\lambda_1, \dots, \lambda_n$ of G, put

$$s^+(G) = \sum_{\lambda_i > 0} \lambda_i^2, \qquad s^-(G) = \sum_{\lambda_i < 0} \lambda_i^2, \qquad \beta(G) = |E(G)| - |V(G)| + 1.$$

A connected graph with $|E(G)| = |V(G)| + 5$ has cyclomatic rank six.

Positive and negative square energies were introduced in connection with spectral coloring bounds by Elphick, Farber, Goldberg, and Wocjan [1]. Liu, Tang, and Zhang proved the conjectured general bound $\min\{s^+(G), s^-(G)\} \geq n - 1$ for connected n-vertex graphs [3]. The conclusion considered here is instead the stronger edge-surplus conjecture of Akbari, Kumar, Mohar, Pragada, and Zhang: a connected n-vertex graph with at least $n+1$ edges should satisfy $s^+(G) \geq n$ [2, Conjecture 1.2]. The present result proves its exact hexacyclic frontier; it does not infer cases with more edges by edge-addition monotonicity.

Theorem 1.1. *Every finite simple connected graph G with $|V(G)| = n$ and $|E(G)| = n+5$ satisfies*

$$\boxed{s^+(G) \geq n}.$$

No strict inequality, equality classification, edge-addition monotonicity, one-edge-subdivision monotonicity, or result for nonsimple realizations is claimed. Human review, independent verification, peer review, and publication are likewise not claimed.

2 Analytic foundations

For a finite graph $H = (V, E)$ define

$$\kappa(H) = \min_{\substack{R \succeq 0 \\ R_{vv}=1 \ (v \in V)}} \sum_{uv \in E} \frac{2}{1 - R_{uv}}, \tag{1}$$

where a summand is $+\infty$ if $R_{uv} = 1$.

Lemma 2.1 (DNN witness implication)**.** *For every finite simple graph H,*

$$s^-(H) \le \kappa(H).$$

Consequently, any feasible correlation matrix with objective at most b proves $s^+(H) \ge 2|E(H)| - b$.

Proof. Write the adjacency matrix as $A = P - N$, where $P, N \succeq 0$ are its positive and negative spectral parts, and put $M = N \circ N$. The Schur product theorem gives $M \succeq 0$ and $M \ge 0$ entrywise. Moreover,

$$\langle J, M \rangle = \operatorname{tr} N^2 = s^-(H), \qquad \frac{s^-(H)}{2} = -\sum_{uv \in E(H)} N_{uv} \le \sum_{uv \in E(H)} \sqrt{M_{uv}}.$$

For every feasible R,

$$\langle J, M \rangle - 2 \sum_{uv \in E(H)} (1 - R_{uv}) M_{uv} = \langle R, M \rangle + 2 \sum_{\substack{u<v \\ uv \notin E(H)}} (1 - R_{uv}) M_{uv} \ge 0.$$

Weighted Cauchy–Schwarz and cancellation when $s^-(H) > 0$ give

$$s^-(H) \le \sum_{uv \in E(H)} \frac{2}{1 - R_{uv}}.$$

The zero and infinite-objective cases are immediate. Minimize over R and use $s^+(H) + s^-(H) = \operatorname{tr} A^2 = 2|E(H)|$. □

Lemma 2.2 (Exact path cost)**.** *If a kernel edge with endpoint correlation r is replaced by a path of length $q \ge 1$, its least DNN contribution in excess of its q edges is*

$$f_q(r) = q \tan^2\left(\frac{\arccos((-1)^q r)}{2q}\right).$$

For fixed r and parity, $f_{q+2}(r) \le f_q(r)$, with endpoint values understood in the extended sense.

Proof. Represent the endpoints by unit vectors and alternately negate vectors along the path. The transformed endpoint angle is $\gamma = \arccos((-1)^q r)$. The spherical triangle inequality and convexity of $\sec^2(x/2)$ make q equal angles optimal, giving $q \sec^2(\gamma/(2q)) = q + f_q(r)$. With $z = \gamma/(2q)$,

$$\frac{d}{dq}\left[q \tan^2\left(\frac{\gamma}{2q}\right)\right] = \tan z \sec^2 z \, (\sin z \cos z - 2z) \le 0.$$

One-sided limits handle endpoint cases. □

Lemma 2.3 (One-vertex additivity)**.** *If $H = H_1 \vee H_2$ is a one-vertex sum, then $\kappa(H) = \kappa(H_1) + \kappa(H_2)$. In particular, $\kappa(T) = |E(T)|$ for every finite tree T.*

Proof. Restriction gives one inequality. For the other, identify the common unit vector in minimizing Gram representations and place the two perpendicular components in mutually orthogonal subspaces. The objectives add. Every tree edge costs at least one, and opposite unit vectors on the two bipartition classes attain one on each edge. □

3 Block decomposition

We use the standard block convention: the blocks of a connected graph are its maximal 2-connected subgraphs together with its bridge copies of K_2. A *positive-rank cyclic block* is a block B with $\beta(B) > 0$. Thus "a unique positive-rank cyclic block" does not mean "one nontrivial block": attached trees contribute nontrivial K_2 blocks.

Lemma 3.1 (Cyclic-block rank additivity)**.** *The cyclomatic rank of a connected graph is the sum of the cyclomatic ranks of its positive-rank cyclic blocks.*

Proof. Following the block–cut tree, adjoining a block B at one cut vertex adds $|V(B)|-1$ vertices and $|E(B)|$ edges, and hence adds $|E(B)|-|V(B)|+1 = \beta(B)$ to the rank. Bridge blocks contribute zero. □

Thus the positive cyclic-block ranks of a rank-six graph form exactly one of

$$1^6, \quad 2+1^4, \quad 2+2+1+1, \quad 2+2+2, \quad 3+1+1+1, \quad 3+2+1, \quad 3+3,$$
$$4+1+1, \quad 4+2, \quad 5+1, \quad 6. \tag{2}$$

The first ten alternatives are the multiblock branch; the final alternative has one positive-rank cyclic block. This is a structural dichotomy only and does not by itself prove either branch.

Lemma 3.2 (Form of the unique-block branch)**.** *If a connected graph has a unique positive-rank cyclic block B, then every other block is a bridge. The graph is obtained from B by one-vertex-summing finite rooted trees at vertices of B.*

Proof. Every other block has rank zero. Under the stated block convention, a rank-zero block of a simple graph is a bridge K_2. Delete the node for B from the block–cut tree. Each remaining component is a tree of bridge blocks attached through exactly one cut vertex of B; two attachments to B would create a cycle in the block–cut tree. Expanding its bridge blocks gives the asserted rooted tree. After suppression is reversed, an attachment vertex may be an internal vertex of a replacement path. □

4 Multiblock branch

Proposition 4.1 (Multiblock branch)**.** *Every finite simple connected rank-six graph with at least two positive-rank cyclic blocks satisfies $s^+(G) \geq |V(G)|$.*

The cited owner-exact multiblock audit [5] proves the proposition and supplies its complete repository ledger. Its scope consists of the first ten rows of (3). It separates direct DNN rows, structural pre-sieve rows, and the packet residuals A–I. Shared cuts, repeated and nested owners, positive bridge connectors, and arbitrary rooted branches are assigned before credit is charged. In its structural rank-three packet, the exceptional doubled-C_4 survivor is the full eight-anchor physical automorphism orbit: four anchors lie in switching class 110 and four in class 111. A genuine kernel-isomorphism audit transports the even- connector opening and the branch/interior-owner territories; no switching operation is used. The direct rank-five-equality-plus-triangle boundary is nonstrict and is not promoted to a strict claim. This skeleton cites that internal exposition rather than duplicating its long owner-by-owner argument. A successful ledger replay checks the encoded predicates; it does not by itself establish that every encoded structural lemma is mathematically sound.

5 Unique positive-rank block

Lemma 5.1 (Rank-six suppression)**.** *Let B be a finite simple 2-connected graph with $\beta(B) = 6$. Suppressing every degree-two vertex produces a loopless 2-connected multigraph K of minimum degree at least three such that*

$$|E(K)| = |V(K)| + 5, \qquad 2 \le |V(K)| \le 10.$$

Conversely, B is a simple realization obtained by replacing kernel edges by positive-length, internally disjoint paths meeting only at prescribed branch endpoints.

Proof. Suppression preserves $|E| - |V|$, connectedness, and rank. A loop would come from a cycle meeting the remainder only at one branch vertex, contradicting 2-connectivity. If $k = |V(K)|$, minimum degree three and $|E(K)| = k + 5$ give $3k \le 2k + 10$, so $k \le 10$; loopless 2-connectivity gives $k \ge 2$. Reversing suppression gives the stated path model. Simplicity is a separate required condition, especially in parallel classes. □

Proposition 5.2 (Exact kernel census)**.** *The rank-six suppressed kernels have the following repository-checked distribution.*

$\lvert V(K)\rvert$	2	3	4	5	6	7	8	9	10
$\#K$	1	4	26	84	216	314	325	162	66

There are 1198 *isomorphism classes in total.*

The exact integer census regenerates multiplicity vectors, rejects supports with a cut vertex, canonicalizes under isomorphism, and compares the complete sorted output with `research/fixtures/rank-six-kernels.json`. No floating-point or spectral calculation enters this census. Its reproducible command is listed in Appendix A.

6 Finite-owner interface

Here an *exact DNN certificate of excess at most five* means an explicit finite record which reconstructs a positive-semidefinite Gram matrix of unit vectors on the realized core, verifies every shared-endpoint identity and every edge denominator $1 - R_{uv} > 0$, and proves by exact integer, rational, or symbolically justified arithmetic that

$$\sum_{uv \in E} \frac{2}{1 - R_{uv}} \le |E| + 5.$$

A structural owner is admissible only if it has a separately stated proof of the same final spectral conclusion for its exact scope; it is not a DNN certificate merely because a verifier assigns it a target key. Numerical optimization output, a symbolic tag without checked identities, or a stored decimal approximation is not an exact certificate.

For each parallel class of multiplicity m with o odd replacement paths, the canonical simple lengths are

$$c(m,o) = \begin{cases} (2, \dots, 2), & o = 0, \\ (1, \underbrace{3, \dots, 3}_{o-1}, \underbrace{2, \dots, 2}_{m-o}), & o > 0. \end{cases}$$

Simplicity permits at most one unit path in a parallel class. After permuting parallel physical edges, every realization vector ℓ in the parity row satisfies $c_i \le \ell_i$ and $\ell_i - c_i \in 2\mathbb{Z}_{\ge 0}$.

Proposition 6.1 (Exact-owner lift)**.** *Suppose every physical parity orbit of every kernel in Proposition 5.2 is owned either by an exact coarse DNN certificate at its canonical vector whose fixed branch Gram remains within excess five after exact path elimination under every same-parity coordinate lengthening, or by exact DNN certificates (or explicitly typed structural owners proving the same conclusion) for every target*

$$\mathcal{F}(c) = \{c\} \cup \{c + 2\mathbf{e}_i : 1 \le i \le |E(K)|\}.$$

For a structural owner, assume additionally an exact all-length and rooted-tree lift covering the descendants for which it claims ownership; Lemma 2.2 may be used only for a DNN owner with a retained branch Gram. Assume that regenerated target keys equal the disjoint union of checked final-owner keys, with no missing, duplicate, or extra keys. Then every simple subdivision of every rank-six kernel, with arbitrary finite rooted trees attached at branch or internal subdivision vertices, satisfies $s^+(G) \ge |V(G)|$.

Proof. If $\ell = c$, use the canonical target. Otherwise choose i with $\ell_i \ge c_i+2$ and start from the owner of $c + 2\mathbf{e}_i$. For a DNN owner, retain its branch vectors, replace the internal vectors independently on every lengthened path, and apply Lemma 2.2; the excess cannot increase. A typed structural owner instead uses the all-length and rooted-tree implication required in the premise. Thus in the DNN case the subdivided core B, with L edges, has $\kappa(B) \le L + 5$.

In that DNN case, if attached rooted trees contain t edges, Lemma 2.3 gives $\kappa(G) \le L + 5 + t$. Rank six gives $|V(G)| = L - 5 + t$, and Lemma 2.1 yields

$$s^+(G) \ge 2(L + t) - (L + 5 + t) = L - 5 + t = |V(G)|.$$

In the structural case, the all-length and rooted-tree conclusion stipulated in the premise gives the same inequality directly. Different frontier targets may use different Gram matrices. No parity-changing subdivision or spectral monotonicity is used. □

The orders-2–10 theorem master verifies this premise over the exact partition $K1$–$K645$, $K646$–$K970$, $K971$–$K1132$, and $K1133$–$K1198$. Its four owners cover orders 2–7, 8, 9, and 10, respectively, and its regenerated counts are $1, 4, 26, 84, 216, 314, 325, 162, 66$. The master authenticates committed canonical exact-execution records for every child, checks the analytic lift separately, and rejects missing, overlapping, widened, or status-only owners.

7 Assembly

Proof of Theorem 1.1. By Lemma 3.1, the positive block ranks have exactly the eleven forms in (3). If there are at least two positive-rank cyclic blocks, apply Proposition 4.1. Otherwise there is one rank-six cyclic block. Lemma 3.2, Lemma 5.1, Proposition 5.2, the verified orders-2–10 owner partition, and Proposition 6.1 prove the second branch. The all-connected master checks that the two branch predicates are disjoint and exhaustive and that both owners state the same nonstrict conclusion. This proves the theorem. □

A Reproducibility ledger

A.1 Commands for present repository artifacts

From the repository root, the kernel census and multiblock ledger can be replayed in normal and optimized modes:

```
python3 research/rank-six-kernel-census-verifier.py
python3 -O research/rank-six-kernel-census-verifier.py
python3 research/doubled-c4-exceptional-owner-transport-verifier.py
python3 -O research/doubled-c4-exceptional-owner-transport-verifier.py
python3 research/hexacyclic-multiblock-ledger-verifier.py
python3 -O research/hexacyclic-multiblock-ledger-verifier.py
```

The complete single-block and all-connected theorem owners are audited with

```
python3 research/rank-six-order2-10-master-verifier.py
python3 -O research/rank-six-order2-10-master-verifier.py
python3 research/hexacyclic-all-connected-master-verifier.py
python3 -O research/hexacyclic-all-connected-master-verifier.py
```

A.2 Frozen theorem-owner identities

The theorem conversion was completed at committed revision `3ac4e68d2ef30b28925d5c30563b5e66ebfae4c0`. The source and canonical-output SHA-256 values below describe the current repaired artifact set; they are content identities, not values attributed to that historical revision:

owner	source SHA-256	output SHA-256
order nine	`d11b6da30ccff5905`	`dd6a5a1e9b905955`
order ten	`4cb22611448755d1`	`767bbf4dd142afd7`
orders 2–10 master	`21aabed4eee24f36`	`e32149c665638e1d`
all-connected master	`e0f9162941cef63b`	`f244139b70ebdfb2`

The displayed values are abbreviated for typesetting. Their complete values, which are the values checked by the verifiers and the reproduction record, are respectively

```
d11b6da30ccff5905cf851d33fc6a5d0cfc6650f3e8317224ddc0d8a2e4cd3d9
dd6a5a1e9b905955affde5fa64e0b8187dc39ad35c02ab007172da3358dff4f4
4cb22611448755d1eea984271251b542ab96f519cc052f3aeda8d56c04c61819
767bbf4dd142afd7eb5a45dffeb738734948412ca53358c40d3655b28403725e
21aabed4eee24f36dc9ddf0460423d0fdd6d20a56e80a60ffed69c796c163bf1
e32149c665638e1d9cd08f8050cd4a7c7cae9f76c51b49f9cda3e0cfb2645834
e0f9162941cef63b1eea3e2268343db3e6cc74b27f6976a01a97d8e83ede6997
f244139b70ebdfb2d375389a1b72c1c8d80bdee5980b78d606b475f0e53bfa8a
```

For the all-connected master, the canonical manifest-plus-text output produced by `--print-manifest` has SHA-256

```
f5494593c500cde76de0ec09f48c207ef94f3be4cdfa91baac2ddcea213d7a6d
```

The committed child-execution evidence has SHA-256

```
bbbb8eff45f8009dfb49e926c4952ae0b231ed6e02e708bddfb13b4aac17bb5d
```

Normal and optimized execution on Python 3.12.3 produced byte-identical orders-2–10 and all-connected outputs with the complete output digests listed above.

A.3 Build

This manuscript is compiled from the repository root with

```
bash scripts/build-paper.sh all-hexacyclic-graphs
```

Build success checks only the LaTeX artifact; the theorem evidence is supplied by the owner chain above, not by typesetting.

B Computational certificate specification

This appendix specifies the formats and acceptance algorithm used by the order-nine and order-ten finite owners. The final theorem owners require full manifest coverage and exact final ownership; an `unresolved` record is a valid encoding but is never an accepting proof record.

B.1 Common conventions and target keys

All integers below are mathematical integers, not machine words. A *uvarint* writes a nonnegative integer in base 128, least significant group first, with bit 7 set exactly when another group follows. A signed integer z is first mapped to $2z$ for $z \geq 0$ and to $-2z - 1$ for $z < 0$, then written as a uvarint. Multi-byte integers are therefore independent of host endianness. The final decoders reject a uvarint with a redundant final zero group by requiring decode–re-encode byte equality, so two encodings of one certificate cannot be accepted.

The frozen kernel fixture, in its listed order, regenerates the residual source stream. A source index j denotes row j of that stream (zero based), not a kernel identifier stored in a pack. The verifier reconstructs from j the kernel, its ordered physical paths, the parity row, canonical lengths, and the $p + 1$ frontier labels

$$C, X_0, \ldots, X_{p-1}, \qquad p = 14 \text{ for order nine}, \quad p = 15 \text{ for order ten}.$$

Thus an exact target key is the pair (j, C) or (j, X_i). A pack owns a half-open source interval $[a, a + r)$, encoded by uvarints a, r. Keys are never accepted from asserted counts: the master regenerates them and compares sets.

Every stored stereographic parameter vector has dimension $d - 1$, where $d \in \{9, 10\}$. One positive common denominator D precedes a witness and each coordinate is a signed-varint numerator n, denoting n/D. Vector and path counts are implicit in the regenerated source row; no untrusted dimension field controls allocation or verification.

B.2 The `R9G1` wire format

After XZ decompression, an order-nine pack is the following byte concatenation:

```
"R9G1" | fixture_sha256[32] | uvarint(start) | uvarint(count)
       | record[0] | ... | record[count-1].
```

The 32-byte digest is binary, not 64 hexadecimal characters. Here $d = 9$, $p = 14$, and a target bitmap has 15 significant bits. Record tags are

byte	name	body and meaning
0	`unresolved`	empty; owns no target
1	`shared`	one shared rational witness, described below
2	`template`	empty; invokes the exact K971 signed-five-cycle audit
3	`individual`	uvarint bitmap, then one witness per set bit

A shared body is D, nine branch parameter vectors, then, in physical-path order, all canonical internal vectors and all length-plus-two internal vectors. For a canonical path of length q_i, these two lists have respectively $q_i - 1$ and $q_i + 1$ vectors. The same branch vectors are used for all 15 frontiers. An individual body begins with a nonzero bitmap smaller than 2^{15}; for each set bit in increasing target order it stores a fresh D, nine branch vectors, and $q_i - 1$ internal vectors on each path, except that the selected X_i path has $q_i + 1$. The template tag is legal only on the regenerated K971 template row. The decoder rejects bad magic or fixture hash, unknown tags, zero denominators, out-of-range intervals or bitmaps, truncation, template/tag mismatch, and trailing raw bytes.

B.3 The `R10G1` wire format

The order-ten header is identical with magic `R10G1`; here $d = 10$, $p = 15$, and the bitmap has 16 significant bits. Tags 0–3 have the same layout, with tag 3 named `fallback`. The current version also reserves payload-free, source-row-dependent exact recognizers:

byte	name	required independent audit
4	`structural`	the typed structural Gram sieve
5	`atom`	the frozen atom-ledger classification and exact atom audit
6	`balanced`	the balanced rank-one recognizer

Tag 2 invokes only the regenerated K1133 signed-five-cycle row. Empty tags do not mean "trust the classification": their recognizers rebuild the required combinatorics and exact Gram or structural predicate. The `R10G1` decoder rejects the malformed cases listed for `R9G1` and requires the raw input to equal its unique re-encoding. XZ is a transport envelope only; acceptance is defined on the decompressed bytes and the exact records they denote.

B.4 Hardware-independent exact arithmetic

For a parameter row $t = (t_1, \dots, t_{d-1}) \in \mathbb{Q}^{d-1}$ the verifier constructs, without floating point,

$$u(t) = \frac{(1 - \langle t, t\rangle, 2t_1, \dots, 2t_{d-1})}{1 + \langle t, t\rangle}.$$

Python arbitrary-precision integers and normalized `Fraction` values implement the field operations. For consecutive unit vectors x, y in the alternating-sign path chain it requires $1 + \langle x, y\rangle > 0$ and adds

$$\frac{1 - \langle x, y\rangle}{1 + \langle x, y\rangle}.$$

The final comparison is the exact cross-multiplied rational inequality $\sum \text{cost} \le 5$. Endpoint vectors are reconstructed from the common branch list, with the terminal branch negated exactly when prescribed by path parity. Consequently unit norm, shared endpoints, path widths, denominator signs, and the budget are rational identities or integer comparisons. Search seeds, floating-point

optimizer scores, wall-clock times, CPU vector instructions, byte order, and native integer overflow cannot affect acceptance. A conforming alternative verifier may use any big-integer rational library, but must produce the same accepted key set from the same raw bytes. XZ decompression, SHA-256, and Python itself remain implementation dependencies; this arithmetic claim is not a claim of independent software verification.

B.5 Fail-closed verifier algorithm

For each order, the theorem verifier executes the following algorithm. Every failure aborts with nonzero status; no `assert`, status string, or stored total is an acceptance condition.

1. Hash and parse the pinned kernel fixture, verifier sources, recognizer sources, census output, and canonical manifest; reject any unpinned byte sequence or noncanonical manifest JSON.
2. Regenerate the complete ordered residual source stream and, from it, the complete expected target-key set. Compare its count and ordered SHA-256 with the pinned values.
3. Read manifest chunks in increasing half-open ranges. Require paths to remain below the manifest directory, and ranges to be nonempty, contiguous, nonoverlapping, and exactly $[0, N)$. Reject repeated paths.
4. For each chunk, check compressed byte count and SHA-256, decompress one XZ stream, check raw byte count and SHA-256, parse the matching embedded interval, and reject malformed or noncanonical records and trailing bytes.
5. For every rational record, reconstruct every relevant frontier chain and perform the exact arithmetic audit above. For every payload-free record, run its named exact recognizer. Map only successfully checked targets to their typed owner; map no target from an unresolved bit or tag.
6. Regenerate the separately pinned symbolic-owner keys. Check that each is proved by its typed recognizer, that rational and final symbolic ownership are disjoint after the prescribed precedence rule, and that their union equals the expected target set exactly. Explicitly reject missing, duplicate, and extra keys.
7. Emit canonical ASCII JSON containing scopes, dependency digests, mode and owner counts, ordered key digests, exact partition equality, and the finite conclusion. Replay under `python3` and `python3 -O`; require zero exits and byte-identical reports. The orders-2–10 master must then validate, rather than copy, these two order conclusions.

Steps 1–4 alone are a digest audit, not certificate verification. Likewise, successful verification of one chunk proves only the records in that chunk; aggregating receipts does not replace a fresh full-manifest exact replay.

B.6 Segmented reproducibility

Large packs may be replayed independently by manifest chunk. From the repository root, first perform the inexpensive range and byte audit:

```
python3 positive-square-energy/experiments/rank6_order9_pack_auditor.py \
  --manifest positive-square-energy/experiments/rank6_order9_search_manifest.json \
  --digest-only
```

```
python3 positive-square-energy/experiments/rank6_order10_pack_auditor.py \
  --manifest positive-square-energy/experiments/rank6_order10_search_manifest.json \
  --digest-only
```

For each manifest chunk index $i = 0, 1, \ldots, m-1$, use a new output path:

```
python3 positive-square-energy/experiments/rank6_order9_pack_auditor.py \
  --manifest positive-square-energy/experiments/rank6_order9_search_manifest.json \
  --chunk-index i --write-chunk-receipt /tmp/r9-replay/chunk-i.json
python3 positive-square-energy/experiments/rank6_order10_pack_auditor.py \
  --manifest positive-square-energy/experiments/rank6_order10_search_manifest.json \
  --chunk-index i --write-chunk-transcript /tmp/r10-replay/chunk-i.json
```

The literal `i` is replaced by the decimal index. The order-ten command also launches and compares optimized replay; order nine must be run explicitly under both `python3` and `python3 -O` until its auditor provides the same built-in comparison. Each segment authenticates the full manifest and dependency map before auditing its own raw records, so segment boundaries cannot alter certificate semantics. A clean full replay requires all chunks, then the full auditor with no `--chunk-index`; if coverage is incomplete or any target remains unowned, the expected result is nonzero status. Receipts and transcripts are resumability aids only and are not theorem owners.

B.7 Final manifest identities

The frozen kernel-fixture SHA-256 embedded in both streams is

```
5a862a0e9ed5dfe91ff6f8491936c8e775eb39b71619df6b8c2a9be2c4643476
```

The canonical order-nine and order-ten manifest SHA-256 values are

```
8aa9d797d9ed786ad438d6fd685e0ec576247b45c17a14749b76c45eebbe9168
5162243e0535ca41f72fa0c7bd27e6ee240d485567e56b8c8dfbcf3a4ecbf3b6
```

respectively. Complete leaf source maps, compressed/raw chunk hashes, ranges, receipt identities, and owner totals remain in those canonical manifests and their transitive theorem-owner dependencies rather than being duplicated as a mutable typeset table.

C Trusted base and limitations

The analytic trust base consists of the displayed matrix arguments, exact path elimination, standard block theory, the suppression reduction, and every structural lemma imported through the multiblock ledger or a typed finite owner. Those imported repository notes are mathematical dependencies, not external validation. The computer-assisted trust base additionally includes the committed verifier and generator sources, canonical kernel fixtures, every certificate and chunk consumed transitively, decompression and parsing code, exact integer/rational and validated symbolic semantics, canonical labelling, the Python runtime, and executing hardware. Hashes establish byte identity, not truth; normal/optimized agreement detects only some implementation errors; hostile mutations are tests, not a completeness proof; and no independent implementation is claimed. Numerical optimization may discover witnesses but is not a proof object. Rooted-tree attachment means a genuine one-vertex sum; a connector meeting a core twice is outside that lemma. Kernel parallel edges are permitted only before realization; the graph to which the trace identity is applied remains simple.

AI disclosure and nonpublication notice

This manuscript was generated by an AI coding and mathematical reasoning system from the repository materials cited below. No human authorship, human review, independent verification, peer review, acceptance, publication, or external announcement is claimed.

References

[1] C. Elphick, M. Farber, F. Goldberg, and P. Wocjan, *Conjectured bounds for the sum of squares of positive eigenvalues of a graph*, Discrete Math. 339 (2016), 2215–2223.

[2] S. Akbari, H. Kumar, B. Mohar, S. Pragada, and S. Zhang, *Refinement of a conjecture on positive square energy of graphs*, arXiv:2506.07264, 2025.

[3] Y. Liu, Q. Tang, and S. Zhang, *The positive and negative square-energy conjecture*, arXiv:2607.18031, 2026.

[4] *LaTeX exposition modules for the rank-six synthesis*, repository proof note, 2026; `positive-square-energy/hexacyclic-general/latex-exposition-modules.md`.

[5] *Hexacyclic multiblock theorem: exhaustive ledger and hostile audit*, repository proof note, 2026; `positive-square-energy/hexacyclic-general/multiblock-items1-7-combined-theorem-audit.md`.

[6] *Audit of the rank-six single-block implication*, repository proof note, 2026; `positive-square-energy/hexacyclic-general/rank-six-single-block-implication-audit.md`.

[7] *Rank-six orders 2–10 master architecture*, promotion-gated repository note, 2026; `positive-square-energy/hexacyclic-general/rank-six-order2-10-master-architecture.md`.

[8] *Order-ten rank-six master architecture*, completion-gated repository note, 2026; `positive-square-energy/hexacyclic-general/order-ten-rank-six-master-architecture.md`.